Kathy Schuler
02/25/2026
IT FDN 130 GP
Assignment 7

Functions

Introduction

This assignment investigates when to use a User-Defined Function and examines the differences between Scalar, Inline, and Multi-Statement Functions.

When to use a SQL User-Defined Function (UDF)

User-Defined Functions (UDFs) can be created by a developer to perform a specific task. Like a recipe, it can be saved and reused when it's needed. A UDF makes a SELECT statement cleaner and easier to read. The function can read and calculate data, but it does not change the underlying database. Parameters are entered between the parentheses. In this example, the UDF calculates the addition of two values:

```
CREATE FUNCTION dbo.AddValues (@Value1 FLOAT, @Value2 FLOAT)
RETURNS FLOAT
AS
  BEGIN
    RETURN (SELECT @Value1 + @Value2);
  END
GO
```

Figure 1: Example of a UDF.

To call this UDF:

```
SELECT dbo.AddValues (4, 5);
```

Figure 2: The SELECT statement for this example.

The results for this example are 9:

Results | Messages

	(No column name)
1	9

Figure 3: The results for this example.

Differences between Scalar, Inline, and Multi-Statement Functions

A Scalar UDF returns a single value (as in the example above) such as a string, integer, or date. It requires a BEGIN and END block. Entering the two-part name (ex: dbo.AddValues) ensures SQL knows where to look for the schema. A Scalar UDF can be used in the SELECT list or in the WHERE clause to transform a single data point.

An Inline Table-Valued Function (iTVF) returns a table. It does not use a BEGIN and END block. It contains a single SELECT statement and is used in the FROM clause. A parameter is entered to match against the data. In this example, the iTVF takes a CategoryID and returns the products belonging to it along with their prices:

```
GO
CREATE FUNCTION fProductsByCategoryID(@ID INT) -- the parameter @ID INT is the input
RETURNS TABLE -- tells SQL to expect rows and columns
AS
RETURN ( -- no BEGIN or END
    SELECT ProductName, UnitPrice
    FROM vProducts
    WHERE CategoryID = @ID
);
GO
```

Figure 4: Example of an iTVF.

The statement to test the iTVF in a FROM clause:

```
SELECT * FROM fProductsByCategoryID(1);
```

Figure 5: The SELECT statement for this example.

The table displays all the Products with a CategoryID of 1 (beverages):

Results | Messages

	ProductName	UnitPrice
1	Chai	18.00
2	Chang	19.00
3	Guaraná Fantástica	4.50
4	Sasquatch Ale	14.00
5	Steeleye Stout	18.00
6	Côte de Blaye	263.50

Figure 6: The results for this example.

A Multi-Statement Table-Valued Function (MSTVF) provides results as a table variable where data is inserted into it. An MSTVF can contain one or more statements. Unlike the Inline version, it requires a defined table structure in the RETURNS clause and a BEGIN and END block to insert data

into the variable. Finally, it requires the RETURN operator. While it can be slower than the Inline version, an MSTVF is useful when multiple steps (such as data filtering or conditional logic) are needed.

In this example of a MSTVF by Esat Erkec (2019), the "return table (@ResultTable) contains ProductName, ScrapQty, ScrapReasonDef, and ScrapStatus columns and this table will be populated and updated in the function body according to the parameter value." The body of the function appears between BEGIN and END. The RETURN statement is used "in order to return the output table" (Erkec, 2019).

```
CREATE FUNCTION dbo.UdfGetProductsScrapStatus
(
@ScrapComLevel INT
)
RETURNS @ResultTable TABLE
(
ProductName VARCHAR(50), ScrapQty FLOAT, ScrapReasonDef VARCHAR(100), ScrapStatus
VARCHAR(50)
) AS BEGIN
        INSERT INTO @ResultTable
            SELECT PR.Name, SUM([ScrappedQty]), SC.Name, NULL
                FROM [Production].[WorkOrder] AS WO
                        INNER JOIN
                        Production.Product AS PR
                        ON Pr.ProductID = WO.ProductID
                        INNER JOIN Production.ScrapReason AS SC
                        ON SC.ScrapReasonID = WO.ScrapReasonID
                WHERE WO.ScrapReasonID IS NOT NULL
                GROUP BY PR.Name, SC.Name
UPDATE @ResultTable
            SET ScrapStatus =
            CASE WHEN ScrapQty > @ScrapComLevel THEN 'Critical'
            ELSE 'Normal'
            END

RETURN
END
```

Figure 7: Creating a multi-statement table-valued function (MSTVF). Adapted from SQL Server multi-statement table-valued functions, by Esat Erkec, 2019, SQL Shack. (https://www.sqlshack.com/sql-server-multi-statement-table-valued-functions/).

The statement to test this MSTVF:

```
SELECT
    ProductName AS [Product Name],
    ScrapQty AS [Scrap Quantity] ,
    ScrapReasonDef AS [Scrap Reason],
    ScrapStatus AS [Scrap Status]
FROM dbo.UdfGetProductsScrapStatus(200)
```

Figure 8: Executing a multi-statement table-valued function. Adapted from SQL Server multi-statement table-valued functions, by Esat Erkec, 2019, SQL Shack. (https://www.sqlshack.com/sql-server-multi-statement-table-valued-functions/).

The results of this MSTVF:

Results | Messages

	Product Name	Scrap Quantity	Scrap Reason	Scrap Status
1	BB Ball Bearing	35	Brake assembly not as ordered	Normal
2	Blade	206	Brake assembly not as ordered	Critical
3	Chain Stays	3	Brake assembly not as ordered	Normal
4	Down Tube	2	Brake assembly not as ordered	Normal
5	Front Derailleur	53	Brake assembly not as ordered	Normal
6	Head Tube	26	Brake assembly not as ordered	Normal
7	HL Fork	26	Brake assembly not as ordered	Normal

Figure 9: Executing a multi-statement table-valued function. Adapted from SQL Server multi-statement table-valued functions, by Esat Erkec, 2019, SQL Shack. (https://www.sqlshack.com/sql-server-multi-statement-table-valued-functions/).

Summary

In conclusion, choosing when to use a Scalar, Inline, or Multi-Statement function depends on the desired output and the complexity of the task. A Scalar UDF is appropriate when a single data point is required. Inline Table-Valued functions deliver faster results because they are set-based (and avoid row-by-row processing). While more intensive, Multi-Statement Table-Valued functions provide the structure for processing complex, multi-step operations.

Reference (all links go to external sites)

Erkec, E. (2019, September 11). SQL Server multi-statement table-valued functions. SQL Shack. https://www.sqlshack.com/sql-server-multi-statement-table-valued-functions/